ALGORITHM DESIGN SPECIFICATION: OPTIMAL EXTRACURRICULAR SCHEDULE FOR TRAINING POINT ACCRUAL

Course: Computational Thinking
Institution: Faculty of Information Technology, University of Science, VNU-HCM

---

1. PROBLEM FORMULATION

1.1. Formal Problem Statement

> **"Formulate an optimal weekly schedule of extracurricular activities yielding Student Training Points (ĐRL), where each activity possesses a defined duration and location, such that the total effective Training Points earned by the student are maximized—strictly adhering to the respective upper bounds across the five designated criteria as well as the global semester ceiling—without conflicting with the student's pre-scheduled academic commitments and while accommodating inter-campus transit requirements."**

1.2. Academic Context & Practical Motivation

At the University of Science, Vietnam National University Ho Chi Minh City (HCMUS), Student Training Points (*Điểm Rèn Luyện - ĐRL*) serve as a statutory assessment metric evaluated on a semester basis to reflect a student's civic, academic, and extracurricular conduct. Under university regulations, this evaluation is governed by **five fundamental criteria**:

- **Criterion 1 (Academic Awareness & Learning Attitude):** Capped at 30 points.
- **Criterion 2 (Compliance with University Regulations & Civic Policies):** Capped at 20 points.
- **Criterion 3 (Participation in Socio-Political, Cultural, and Athletic Activities):** Capped at 25 points.
- **Criterion 4 (Civic Responsibility & Community Engagement):** Capped at 25 points.
- **Criterion 5 (Leadership & Class/Union Management):** Capped at 30 points.
- **Global Statutory Semester Ceiling:** Strictly bounded at a maximum of **100 points**.

In the daily reality of undergraduate students, several critical scheduling constraints emerge:

1. **Inflexible Pre-Scheduled Commitments:** Students maintain mandatory schedules consisting of lectures, laboratory practicals, formal examinations, and essential personal duties. These time intervals are **strictly non-negotiable** (they cannot be rescheduled, postponed, or omitted).
2. **Concurrent Extracurricular Overlap & Inter-Campus Transit:** During peak academic intervals (such as Youth Month, Welcome Campaigns for Freshmen, or specialized hackathons), multiple activities take place concurrently across two geographically separated facilities: **Campus 1** (District 5, Ho Chi Minh City) and **Campus 2** (Linh Trung Ward, Thu Duc City). A student cannot attend overlapping events, and transitioning between campuses necessitates an indispensable transit buffer of at least 60 minutes.
3. **Diminishing Marginal Returns Under Category Caps (Submodular Marginal Gain):** Because each criterion enforces a rigid upper threshold, once a student saturates a specific criterion (e.g., reaching 25/25 points in Criterion 3), attending further activities within that category yields **zero marginal points** ($\Delta p = 0$), resulting in redundant time expenditure.
4. **System Objective:** Synthesize an optimal, clash-free extracurricular itinerary that maximizes net points earned toward the statutory ceiling without squandering time on saturated categories or redundant activities.

2. PROBLEM DECOMPOSITION

In accordance with the principles of Computational Thinking, an intractable real-world problem is decomposed into **three core constituent sub-problems**:

1. **Compiling the candidate pool of extracurricular activities with defined durations.**
2. **Maximizing the effective Training Points accrued within the week, subject to category-specific caps and a global semester cap.**
3. **Enforcing non-conflict constraints with the student's fixed schedule and accommodating inter-campus transit buffers.**

Detailed Analysis of Decomposed Sub-Problems:

2.1. Sub-Problem 1: Compiling the Candidate Pool of Extracurricular Activities with Defined Durations

- **Problem Essence:** Throughout the academic semester, numerous student unions, academic clubs, and institutional departments broadcast an abundance of seminars, volunteer campaigns, and academic competitions.
- This sub-problem focuses on surveying, cataloging, and formalizing the set of extracurricular events occurring within the week that grant Training Points.
- Each candidate activity is characterized by concrete quantitative attributes:
  - **Temporal Footprint:** Concrete start time, end time, and duration (or an explicit submission deadline for asynchronous online assessments). The duration quantifies the "temporal cost" the student must invest.
  - **Nominal Points:** The advertised point incentive awarded upon successful participation.
  - **Category Classification:** The specific statutory criterion among the five designated categories to which the event belongs.
- **Significance:** This step formalizes the candidate decision space—delineating the exact set of selectable items available for inclusion in the student's timetable.

2.2. Sub-Problem 2: Maximizing the Effective Training Points Accrued (Subject to Category-Specific Caps and Global Cap)

- **Problem Essence:** The principal objective is to discover a selection of events that yields the maximum possible cumulative point gain over the course of the week.
- However, points cannot be simply accumulated monotonically; they are strictly bounded by a **Double-Capping Regulatory Model**:
  - **Category-Specific Caps:** Each of the 5 criteria has an independent maximum threshold. When a student saturates a category, any additional participation in that category yields **0 net points**.
  - **Global Semester Cap:** Cumulative recognized points are strictly bounded at **100 points**. Once the aggregate reaches 100, no further point gain can be realized regardless of category.
- Consequently, the marginal point gain of any event exhibits a submodular, diminishing-returns property that depends directly on the student's current point profile. The system must accurately compute the marginal gain of each event, prioritizing activities that fulfill deficit categories and discarding activities in saturated ones.
- **Significance:** This step defines the objective function and the optimization benchmark: maximizing actual net point yield rather than nominal points.

2.3. Sub-Problem 3: Enforcing Non-Conflict Constraints with Fixed Schedules and Accommodating Transit Buffers

- **Problem Essence:** A student's schedule is governed by strict temporal and spatial feasibility constraints:
  - **Preservation of Fixed Commitments:** Formal lectures, lab sessions, and exams cannot be displaced. Extracurricular events must never intersect these pre-existing commitments.
  - **Mutual Exclusivity of Selected Events:** A student cannot attend two overlapping extracurricular events simultaneously.
  - **Inter-Campus Transit Buffers:** Moving between Campus 1 (District 5) and Campus 2 (Thu Duc City) requires an essential buffer of at least 60 minutes. If consecutive events occur across different campuses, they must be separated by at least 60 minutes. Any candidate event failing this transit threshold is infeasible.
- **Significance:** This step defines the feasible solution space: ensuring that the resulting schedule is physically executable, conflict-free, and legally compliant.

3. PATTERN RECOGNITION

3.1. Formal Classification: Variant of 0/1 Knapsack with Interval Conflicts and Category Caps

Synthesizing the three decomposed components directly reveals an elegant isomorphism with classic paradigms in Computer Science:

- **Component 1 (Candidate Activities with Durations)** $\longrightarrow$ Defines the set of **Selectable Items** with associated resource costs (time footprint).
- **Component 2 (Maximizing Net Accrual under Double Capping)** $\longrightarrow$ Defines the **Objective Function to Maximize Value**, augmented with category compartment caps and an overarching global ceiling.
- **Component 3 (Temporal Non-Conflict & Transit Constraints)** $\longrightarrow$ Defines **Knapsack Capacity & Conflict Graph Invariants**, specifically mapping to Interval Graphs with adjacency determined by temporal overlap and transit deficits.

Therefore, the problem is formally recognized as a **Variant of the 0/1 Knapsack Problem with Interval Conflicts and Category Compartment Caps**

3.2. Structural Isomorphism Mapping Table

Classic 0/1 Knapsack Element	Extracurricular Schedule Mapping
Items ($i \in \{1, \dots, n\}$)	**Candidate Extracurricular Events**. Decision variable $x_i \in \{0, 1\}$ denotes whether event i is selected ($x_i = 1$) or omitted ($x_i = 0$).
Item Value (v_i)	**Nominal Training Points** p_i. However, the effective marginal gain Δp_i is submodular, depending on accumulated points in Category c: $\Delta p_i = \min\left(p_i,\ \max(0, \text{Cap}_c - \text{current_points}[c]),\ \max(0, 100 - \sum \text{current_points})\right)$ **Variable Definitions:** • Δp_i: Net effective points accrued from selecting event i. • p_i: Nominal points advertised for event i. • $\text{current_points}[c]$: Points accumulated in Category c prior to selecting event i. • $\text{Cap}_c - \text{current_points}[c]$: Remaining capacity in Category c before saturation. • $100 - \sum \text{current_points}$: Remaining capacity before reaching the global semester ceiling of 100.
Weight / Resource Cost (w_i)	**Temporal Footprint** $[s_i, e_i]$. The available free intervals across the student's weekly timetable represent the aggregate "Knapsack Capacity".
Conflict Invariants	**1D Interval Graph Conflicts:** Events i and j cannot be simultaneously selected if their time intervals overlap or if they violate the 60-minute inter-campus transit buffer.
Category Compartment Caps	**Five Statutory Criteria:** The knapsack is partitioned into 5 independent compartments, each governed by an upper threshold ($\text{Cap}_1 \dots \text{Cap}_5$). Saturated compartments yield zero marginal value for additional items.
Global Capacity Ceiling	**Global Cap of 100 Points:** The cumulative value across all five compartments cannot exceed 100 points.

4. ALGORITHM DESIGN: ADVANCED BRANCH AND BOUND

To solve the scheduling problem to global optimality while guaranteeing instantaneous response times, the system implements an **Advanced Branch and Bound with Next Compatible Jump**.

4.1. Algorithmic Principles & Precomputations ($O(1)$ Jump Search Tree)

Rather than traversing a conventional binary decision tree (which examines each event sequentially via $i \to i + 1$ and requires nested loops to verify temporal compatibility), the algorithm establishes an $O(1)$ **Jump Search Tree** bolstered by offline precomputations:

1. **Chronological Sorting:**
    - All feasible candidate activities are pre-sorted in ascending order: `(start_time, end_time)`.

2. **Precomputed $O(1)$ Next Compatible Jump Array (`precomputed_next`):**
   - For each event i, the algorithm precomputes the smallest index $j > i$ such that event j is strictly compatible with event i, accounting for the necessary transit buffer:

$$s_j \geq e_i + \text{TravelBuffer}(\text{loc}_i, \text{loc}_j)$$

     - **Variable Definitions:**
       - s_j: Start time of candidate event j ($j > i$).
       - e_i: End time of event i.
       - $\text{loc}_i, \text{loc}_j$: Geographic locations of events i and j (CS1, CS2, or ONLINE).
       - $\text{TravelBuffer}(\text{loc}_i, \text{loc}_j)$: Transit buffer required between locations:

$$\text{TravelBuffer}(\text{loc}_i, \text{loc}_j) = \begin{cases} 60 \text{ minutes} & \text{if } \{\text{loc}_i, \text{loc}_j\} = \{\text{CS1}, \text{CS2}\} \\ 0 \text{ minutes} & \text{if identical campus or either event is ONLINE} \end{cases}$$

   - Stored in array `precomputed_next[i] = j` . When event i is selected, the search instantly **jumps to index j in $O(1)$ time**, eliminating 100% of nested temporal collision checks during recursion.
3. **Precomputed Suffix-Sum Potential Matrix (`remaining_by_cat`):**
   - For each Category $c \in \{1 \dots 5\}$, nominal points are accumulated in reverse order from the end of the candidate list down to index i:

$$\text{remaining_by_cat}[c][i] = \sum_{k=i}^{n-1} \left(p_k \cdot \mathbb{I}(\text{event}_k \in \text{Category } c) \right)$$

     - **Variable Definitions:**
       - c: Target statutory criterion ($c \in \{\text{dieu_1}, \dots, \text{dieu_5}\}$).
       - i: Current event index in the sorted candidate pool ($0 \leq i < n$).
       - k: Iterative index traversing future candidate events from i to $n-1$.
       - p_k: Nominal point value of event k.
       - $\text{remaining_by_cat}[c][i]$: Theoretical upper bound on remaining points obtainable from Category c from index i onward.
   - Consequently, computing the global theoretical upper bound at any search node executes in $O(1)$ **lookup time** instead of requiring an $O(n)$ iteration.
4. **Greedy Warm-Start Initialization:**
   - Prior to launching depth-first recursion, a lightweight $O(n)$ greedy heuristic pass is executed to discover an initial feasible solution.
   - Setting $\text{best_score} = \text{GreedyScore}$ establishes an aggressive lower bound from the outset, enabling immediate branch pruning at shallow depths of the search tree.

4.2. Four-Tier Cascading Pruning Mechanics

The recursive search function `search(current_idx, current_selected, current_points, score)` applies a systematic 4-tier filtering funnel to prune non-optimal subtrees:

Tier 0: Global Cap Pruning (Early Termination)

- If $\sum_{c=1}^{5} \text{current_points}[c] \geq 100 \implies$ Invoke `return` immediately!
  - **Variable Definitions:**
    - $\text{current_points}[c]$: Points accumulated in Category c at the current node.
    - $\sum_{c=1}^{5} \text{current_points}[c]$: Total points accrued across all 5 categories.
    - 100: Statutory maximum recognized per semester.
- Reaching the statutory maximum makes further exploration completely redundant, immediately terminating the branch.

Tier 1: Global Upper Bound Pruning (Node Level)

- At the entrance of `search()`, calculate the theoretical maximum points achievable if all remaining events from `current_idx` onward could be selected, bounded by category caps and the global ceiling:

$$\text{Cat_Potential} = \sum_{c=1}^{5} \min\left(\max(0,\ \text{Cap}_c - \text{current_points}[c]),\ \text{remaining_by_cat}[c][\mathbf{current_idx}]\right)$$

$$\text{Global_Slack} = \max\left(0,\ 100 - \sum_{c=1}^{5} \text{current_points}[c]\right)$$

$$\text{Potential} = \min\left(\text{Global_Slack},\ \text{Cat_Potential}\right)$$

$$\text{Upper Bound} = \text{score} + \text{Potential}$$

  - **Variable Definitions:**
    - current_idx: Pointer to the next candidate event to be evaluated ($0 \le \text{current_idx} \le n$).
    - $\text{remaining_by_cat}[c][\text{current_idx}]$: Total points remaining in Category c from current_idx to $n-1$.
    - $\max(0, \text{Cap}_c - \text{current_points}[c])$: Residual capacity in Category c.
    - Cat_Potential: Maximum potential gain bounded individually across all five categories.
    - Global_Slack: Residual capacity before reaching the global statutory limit of 100.
    - Potential: Strict upper bound on achievable points from this branch.
    - score: Cumulative points earned from events selected so far ($\text{score} = \sum_{c=1}^{5} \text{current_points}[c]$).
    - Upper Bound: Theoretical maximum score achievable in this subtree.
    - best_score: Best global score discovered so far across the search space.
- **Pruning Invariant:** If $\text{Upper Bound} \le \text{best_score} \implies$ Trigger an immediate `return` to backtrack, pruning the entire subtree.

Tier 2: Local Look-ahead Pruning (Loop Level)

- Within the loop `for i in range(current_idx, total_candidates)`, the algorithm evaluates selecting event i (implicitly skipping all events from current_idx to $i-1$).
- The residual potential from index i to the end of the array is computed:

$$\text{quick_slack} = \max\left(0,\ 100 - \sum_{c=1}^{5} \text{current_points}[c]\right)$$

$$\text{quick_pot} = \min\left(\text{quick_slack},\ \sum_{c=1}^{5} \min\left(\max(0,\ \text{Cap}_c - \text{current_points}[c]),\ \text{remaining_by_cat}[c][\mathbf{i}]\right)\right)$$

  - **Variable Definitions:**
    - i: Index of the candidate event currently tested in the loop ($i \ge \text{current_idx}$).
    - quick_pot: Potential point gain from index i onward.
    - *Monotonicity Principle:* Because `remaining_by_cat` is monotonically non-increasing with respect to i, if $\text{score} + \text{quick_pot} \le \text{best_score}$ at index i, all subsequent indices $j > i$ are guaranteed to fail as well.
- **Pruning Invariant:** If $\text{score} + \text{quick_pot} \le \text{best_score} \implies$ Execute `break`! This terminates the entire loop early.

Tier 3: Category & Global Cap Pruning (Event Level)

- For individual event i belonging to Category c, compute its net effective point gain:

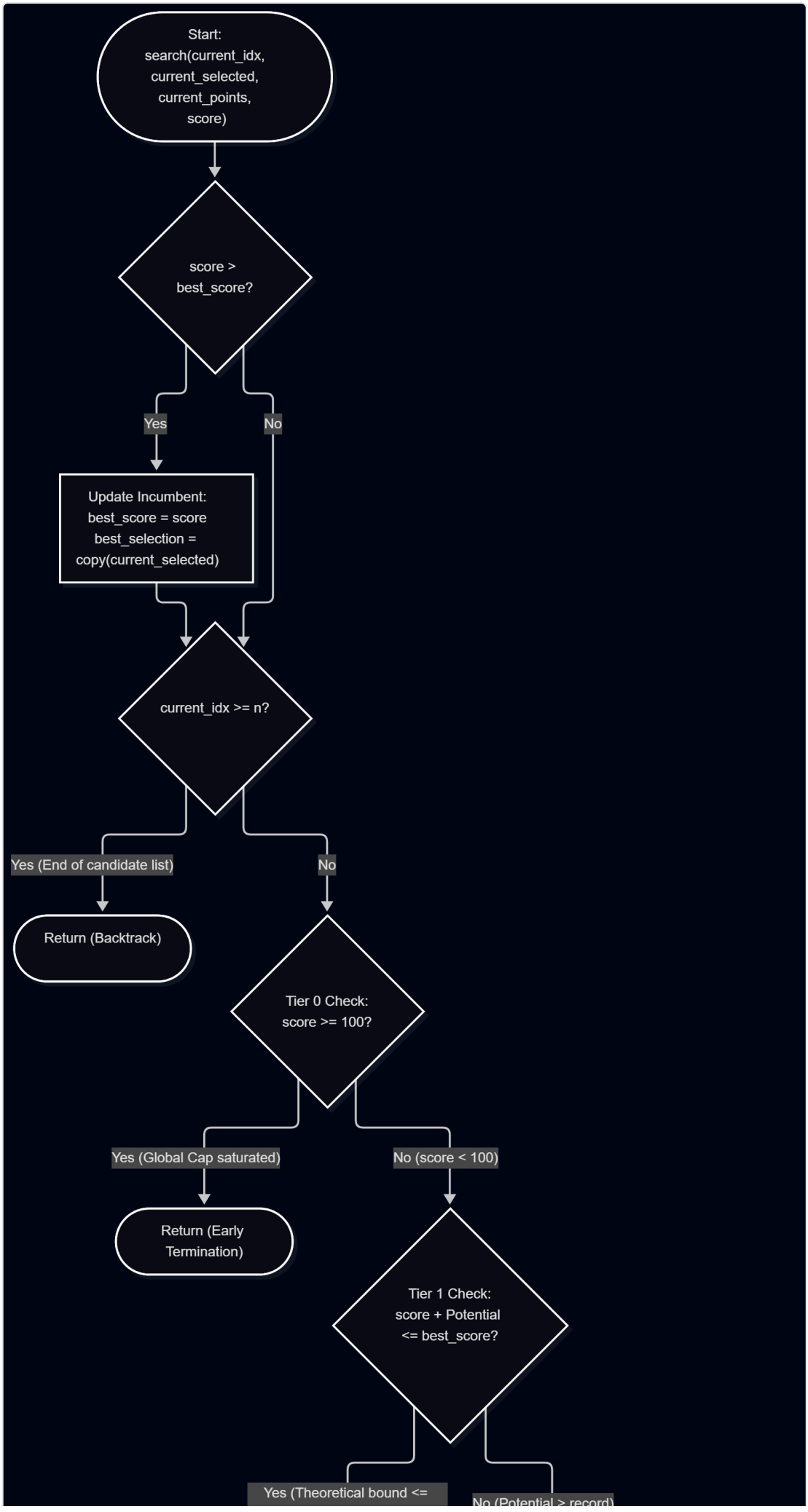
$$\text{gain} = \min\left(p_i,\ \max(0,\ \text{Cap}_c - \text{current_points}[c]),\ \max\left(0,\ 100 - \sum_{k=1}^{5} \text{current_points}[k]\right)\right)$$

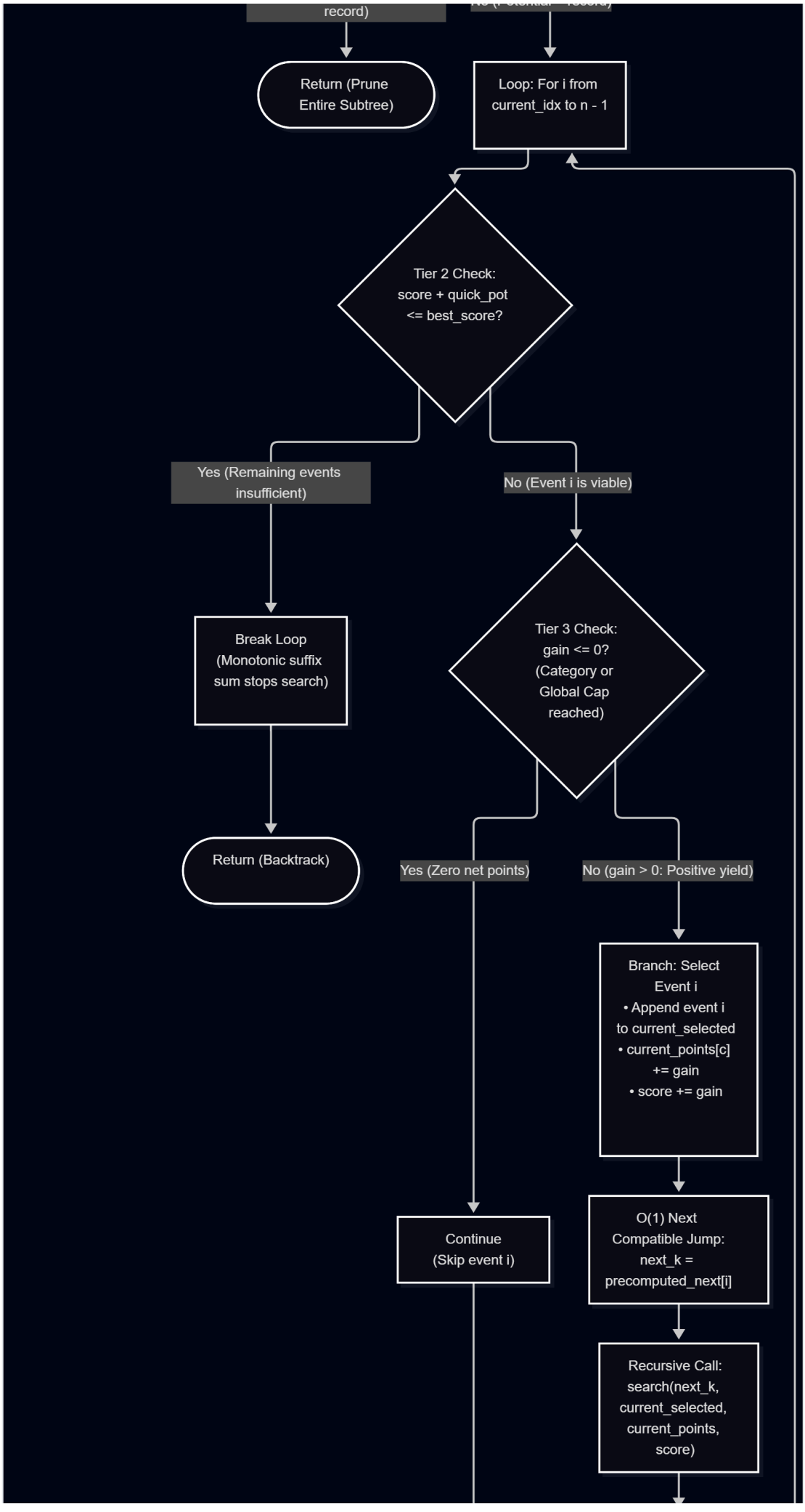
  - **Variable Definitions:**
    - p_i: Advertised point value of event i.
    - c: Statutory category of event i ($c = \text{event}_i.\text{category}$).
    - gain: Actual point increment realized by the student upon attending event i.
- **Pruning Invariant:** If $\text{gain} \le 0$ (Category c is saturated or global cap is reached) $\implies$ Execute `continue` to skip event i, preventing time wastage on zero-yield activities.

- If $\text{gain} > 0 \implies$ Execute an $O(1)$ jump to `next_k = precomputed_next[i]` and recurse: `search(next_k, ..., score + gain)`.

4.3. Algorithmic Activity Flowchart

To visualize the execution pipeline, decision points, and cascading pruning triggers of the Branch and Bound algorithm, the activity flow is formally depicted below:


Start: search(current_idx, current_selected, current_points, score)
score > best_score?
Yes
No
Update Incumbent: best_score = score best_selection = copy(current_selected)
current_idx >= n?
Yes (End of candidate list)
No
Return (Backtrack)
Tier 0 Check: score >= 100?
Yes (Global Cap saturated)
No (score < 100)
Return (Early Termination)
Tier 1 Check: score + Potential <= best_score?
Yes (Theoretical bound <=
No (Potential > record)


record)
Return (Prune Entire Subtree)
Loop: For i from current_idx to n - 1
Tier 2 Check: score + quick_pot <= best_score?
Yes (Remaining events insufficient)
No (Event i is viable)
Break Loop (Monotonic suffix sum stops search)
Tier 3 Check: gain <= 0? (Category or Global Cap reached)
Return (Backtrack)
Yes (Zero net points)
No (gain > 0: Positive yield)
Branch: Select Event i
• Append event i to current_selected
• current_points[c] += gain
• score += gain
Continue (Skip event i)
O(1) Next Compatible Jump: next_k = precomputed_next[i]
Recursive Call: search(next_k, current_selected, current_points, score)



Branch and Bound Algorithmic Activity Flowchart

4.4. Complexity & Empirical Performance Analysis

1. **Space Complexity:**
   - Eliminates large multi-dimensional matrices.
   - Auxiliary memory is strictly limited to 1D vectors of size n: `precomputed_next` (n integers) and `remaining_by_cat` ($5 \times n$ integers).
   - Call stack depth is bounded by n. Each recursive frame maintains only a lightweight 5-element dictionary `current_points`.
2. **Time Complexity:**
   - *Theoretical Worst-Case:* $O(2^n)$ in the absence of pruning.
   - *Empirical Performance in University Scenarios:*
     - Due to the $O(1)$ **jump array**, selecting an event lasting $2-3$ hours bypasses dozens of mutually overlapping events in a single step, limiting effective tree depth.
     - With **Greedy Warm-Start**, the initial incumbent `best_score` is already near-optimal, allowing Tier 1 and Tier 2 pruning to eliminate most of search branches.
     - Actual node evaluations range from **a few hundred to a few thousand nodes**.

5. EXPERIMENTAL VALIDATION & COMPREHENSIVE TESTING SUITE

To substantiate the mathematical correctness, spatial-temporal robustness, and real-time efficiency of the proposed Branch and Bound scheduling engine, a comprehensive testing and benchmarking ecosystem was established under `Backend/Unit_Test/`, comprising three complementary components:

5.1. Automated Unit Testing Suite (`test_branch_and_bound_scheduler.py`)

The automated unit test suite is built on Python's standard `unittest` framework, validating operational invariants and edge cases against database-modeled task structures across seven rigorous test scenarios:

1. **Test Case 1 — Hard Anchor Schedule Protection:**
   - *Objective:* Verify that extracurricular activities overlapping with mandatory academic commitments (lectures, laboratory practicals, or exams) are unconditionally pruned.
   - *Result:* The scheduler strictly eliminates conflicting events while successfully scheduling non-overlapping opportunities, ensuring zero academic schedule disruption.
2. **Test Case 2 — Inter-Campus Transit Invariant Enforcement:**
   - *Objective:* Validate the geographical constraint requiring a minimum 60-minute travel buffer between Campus 1 (District 5) and Campus 2 (Thu Duc City).
   - *Result:* Events separated by only 45 minutes across different campuses are correctly detected as physically infeasible and pruned, whereas same-campus transitions are accepted.
3. **Test Case 3 — Submodular Category Capping (Double Capping):**

- *Objective:* Test marginal point saturation (e.g., student possesses 27/30 points in Criterion 1; an event with 10 nominal points should only yield +3 net points).
- *Result:* Verified that the effective gain is precisely truncated to $\Delta p = 3$ and post-schedule total equals the exact 30-point ceiling.

4. **Test Case 4 — Conflict Resolution Among Competing DRL Activities:**
   - *Objective:* When multiple mutually overlapping extracurricular events compete for the same time window, verify that the algorithm selects the combination yielding the highest net points.
   - *Result:* Confirmed global optimality by prioritizing higher-yield events over lower-yield alternatives.
5. **Test Case 5 — Asynchronous Online Deadline-Only Events:**
   - *Objective:* Handle online quizzes (e.g., MyAloha) lacking explicit start/end timestamps.
   - *Result:* Confirmed automated allocation of a 30-minute execution window immediately preceding the deadline.
6. **Test Case 6 — High-Density Scalability Benchmark ($N = 40$):**
   - *Objective:* Stress-test the search engine with 40 densely packed, competing events across a full 7-day academic week.
   - *Result:* Successfully solved to absolute optimality in $< 10\ \text{ms}$ (well within the $< 200\ \text{ms}$ threshold), confirming real-time responsiveness.
7. **Test Case 7 — Global Ceiling Invariant & Early Stopping (100 Points):**
   - *Objective:* When a student already holds 92 points, ensure the scheduler selects only enough events to achieve the remaining 8 points to reach the 100-point ceiling, and terminates search early.
   - *Result:* The engine selects exactly 2 events totaling 8 net points and stops immediately, protecting the student's remaining personal time from redundant scheduling.

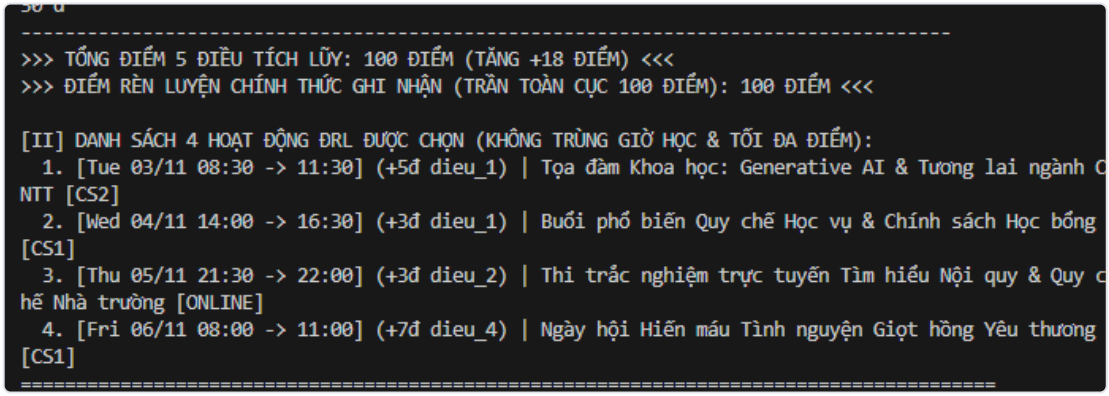
5.2. End-to-End Real-World Simulation Demo (`demo_branch_and_bound.py`)

This script models an end-to-end semester week (November 02, 2026 to November 08, 2026) for an undergraduate student in the Faculty of Information Technology (FIT - HCMUS):

- **Ingested Data Profile:** Consumes heterogeneous task records directly conforming to the PostgreSQL `tasks` database schema, comprising:
  - 5 fixed university lectures across Campus 1 and Campus 2 (Calculus, Programming, Computer Networks, Physical Education, Data Structures).
  - Recurring personal commitments (IELTS preparation block).
  - 8 diverse candidate extracurricular events (scientific seminars, hackathons, blood donation drives, CV workshops, Green Sunday environmental campaigns, and online compliance quizzes).
- **Initial Student Profile:** Simulated with an initial baseline of **82 points** across five categories:
  - Criterion 1: 22/30 (8 points remaining)
  - Criterion 2: 17/20 (3 points remaining)
  - Criterion 3: 18/25 (7 points remaining)
  - Criterion 4: 15/25 (10 points remaining)
  - Criterion 5: 10/30 (20 points remaining)
- **Execution & Output:** The scheduler successfully resolves classroom collisions, enforces the 60-minute travel buffer, accrues exactly **18 points** across deficit criteria to hit the statutory **100-point total**, and outputs an intuitive visual timetable breakdown:



Simulation Demo Results - Category Criteria Balance Verification



Simulation Demo Results - Optimal Scheduled Extracurricular Activities

5.3. Comparative Benchmark & Empirical Performance Evaluation

(`Benchmark_3_Algorithms_Comparison.ipynb`)

A rigorous scientific Jupyter notebook conducting empirical side-by-side benchmarking across **15 standardized test scenarios** (spanning edge cases and scalability suites from $N = 5$ to $N = 40$) comparing three alternative algorithms:

1. **Advanced Branch and Bound** (Proposed solution with $O(1)$ jumps, suffix-sum pruning, and early stopping).
2. **Multi-Dimensional Dynamic Programming (DP)** (Exact 6-dimensional state table with memoization).
3. **Density Greedy Heuristic** (Local greedy selection prioritized by points per minute).

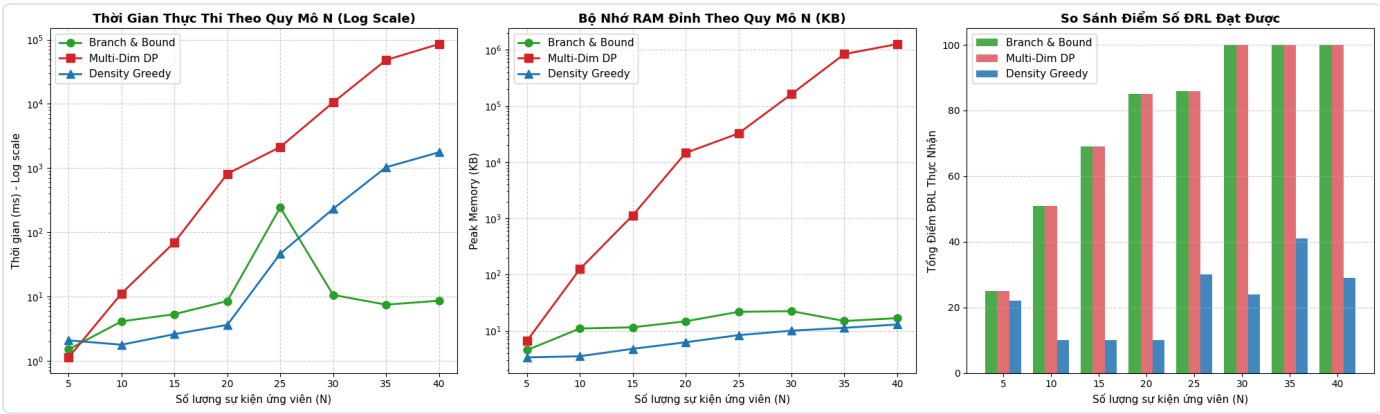
Empirical Benchmark Results Table:

Test Scenario	Advanced Branch & Bound *(Score / Time / Peak RAM)*	Multi-Dimensional DP *(Score / Time / Peak RAM)*	Density Greedy Heuristic *(Score / Time / Peak RAM)*
Case 01: Hard Anchor Clash	**5 pts** / 0.497 ms / 2.77 KB	**5 pts** / 0.494 ms / 5.67 KB	**5 pts** / 0.180 ms / 1.25 KB
Case 02: Campus Travel Constraint	**8 pts** / 0.445 ms / 2.73 KB	**8 pts** / 0.277 ms / 1.93 KB	**8 pts** / 0.214 ms / 1.25 KB
Case 03: Submodular Cap Ceiling	**8 pts** / 0.431 ms / 2.50 KB	**8 pts** / 0.234 ms / 2.06 KB	**8 pts** / 0.267 ms / 1.50 KB
Case 04: Density Greedy Trap	**10 pts** / 0.525 ms / 3.15 KB	**10 pts** / 0.235 ms / 1.89 KB	3 pts / 0.208 ms / 1.35 KB *(Trap!)*
Case 05: Multi-Category Diversity	**45 pts** / 1.112 ms / 3.62 KB	**45 pts** / 0.836 ms / 7.41 KB	40 pts / 1.598 ms / 2.52 KB
Case 06: Online Deadline Quiz	**13 pts** / 0.405 ms / 2.48 KB	**13 pts** / 0.265 ms / 2.46 KB	5 pts / 0.205 ms / 1.35 KB
Case 07: Dense Clash Cluster	**10 pts** / 1.796 ms / 9.05 KB	**10 pts** / 0.924 ms / 3.47 KB	**10 pts** / 0.567 ms / 2.78 KB

Test Scenario	Advanced Branch & Bound *(Score / Time / Peak RAM)*	Multi-Dimensional DP *(Score / Time / Peak RAM)*	Density Greedy Heuristic *(Score / Time / Peak RAM)*
Case 08: Scale $N = 5$	**25 pts** / 1.510 ms / 4.55 KB	**25 pts** / 1.130 ms / 6.64 KB	22 pts / 2.082 ms / 3.37 KB
Case 09: Scale $N = 10$	**51 pts** / 4.114 ms / 11.01 KB	**51 pts** / 11.205 ms / 125.62 KB	10 pts / 1.781 ms / 3.54 KB
Case 10: Scale $N = 15$	**69 pts** / 5.322 ms / 11.58 KB	**69 pts** / 69.896 ms / 1,135.65 KB *(~1.14 MB)*	10 pts / 2.600 ms / 4.79 KB
Case 11: Scale $N = 20$	**85 pts** / 8.525 ms / 14.81 KB	**85 pts** / 815.081 ms / 14,677.75 KB *(~14.68 MB)*	10 pts / 3.627 ms / 6.26 KB
Case 12: Scale $N = 25$	**86 pts** / 243.757 ms / 21.81 KB	**86 pts** / 2,133.546 ms / 32,664.01 KB *(~32.66 MB)*	30 pts / 46.407 ms / 8.40 KB
Case 13: Scale $N = 30$	**100 pts** / 10.627 ms / 22.46 KB	**100 pts** / 10,522.691 ms / 163,297.61 KB *(~163.30 MB)*	24 pts / 232.328 ms / 10.07 KB
Case 14: Scale $N = 35$	**100 pts** / 7.488 ms / 14.98 KB	**100 pts** / 48,189.523 ms / 841,950.12 KB *(~841.95 MB)*	41 pts / 1,031.285 ms / 11.33 KB
Case 15: Scale $N = 40$	**100 pts** / **8.622 ms** / **16.87 KB**	**100 pts** / **85,275.116 ms** / **1,267,083.75 KB** *(~1.27 GB)*	29 pts / 1,762.949 ms / 13.00 KB

Empirical Scalability Visual Comparison:

To synthesize the comparative performance across all three algorithms, the benchmark curves for execution time (log scale), peak memory allocation (log scale), and total earned training points across candidate event scales $N \in [5, 40]$ are visualized below:



Scalability Benchmark Comparison across 3 Algorithms

5.4. Core Empirical Findings & Algorithmic Justification

Synthesizing the empirical data from the benchmark table and the three comparative scalability charts, three core conclusions are established based on actual execution metrics:

1. **Global Optimality & Double Capping Invariant Enforcement:**
   - **Absolute Mathematical Optimality:** Both **Branch & Bound** and **Multi-Dimensional DP** consistently achieve **100% identical optimal point scores** across all benchmark test scenarios. In large-scale benchmarks (such as $N = 40$), both algorithms achieve exactly **100 points**—strictly complying with the statutory semester ceiling established by the HCMUS Regulations on Student Training Points—thereby mathematically validating the exactness and correctness of both paradigms.
   - **Failure of Greedy Heuristics:** In stark contrast, the **Density Greedy** heuristic frequently becomes trapped in suboptimal local extrema due to its myopic focus on short-term point density per minute. Under scaled candidate pools ($N \geq 10$), this myopic selection precipitates severe point deficits ranging from 30% **to over** 70% relative to global optimality (e.g., at $N = 40$, Greedy accrues only **29/100 points**, a devastating 71% **point loss**).
2. **Computational Execution Efficiency & Pruning Breakthrough:**
   - **Combinatorial Explosion of DP:** Multi-Dimensional DP exhibits exponential runtime explosion as problem dimensions expand, surging from sub-millisecond execution to $85,275.116\ \text{ms}$ **(**$\approx 85.28\ \text{seconds}$**)** at $N = 40$. This

bottleneck stems from the severe state-space explosion inherent in tracking continuous 6-dimensional submodular states.

- **Ultra-Fast Stability of Branch & Bound:** Branch & Bound maintains remarkable runtime efficiency and stability across all scales, solving the $N = 40$ scenario in merely $8.622\ \text{ms}$—an acceleration factor of nearly $9,890\times$ over DP. This breakthrough is driven by **Tier 0: Global Cap Pruning** coupled with the precomputed $O(1)$ **jump array**: the instant a valid schedule path reaches the statutory 100-point ceiling, search tree expansion is immediately truncated, entirely bypassing redundant branch explorations.

3. **Memory Consumption & Resource Safety:**
   - **Prohibitive Memory Footprint of DP:** Multi-Dimensional DP incurs massive memory consumption, rapidly escalating from several kilobytes to hundreds of megabytes, and culminating in $1,267,083.75\ \text{KB}$ **(**$\approx 1.27\ \text{GB}$ **of RAM)** at $N = 40$ to maintain its dynamic memoization table. In multi-tenant cloud environments or backend web services, this magnitude of consumption presents acute Out-Of-Memory (OOM) crashing risks.
   - **Lightweight Footprint of Branch & Bound:** Branch & Bound requires merely a few dozen kilobytes of auxiliary memory ($16.87\ \text{KB}$ at $N = 40$), confined strictly to recursion depth on the execution **Call Stack**. This minuscule footprint ensures extreme system reliability, deterministic stability, and negligible server overhead under concurrent workloads.